Technical Specification: std::vector Clone

> **Objective:** Develop a dynamic array in C++ (template) that manages its own heap memory. This project is the mandatory rite of passage to master granular memory management, data contiguity, and low-level performance.

1. Data Architecture

The class encapsulates a raw heap pointer while exposing a safe interface to the user. No dependencies on the standard library for allocation (except `std::move` or basic types) are allowed.

Component	Type	Role & Responsibility
`m_Data`	`T*`	Raw pointer to the memory block allocated on the heap. Points to the first element.
`m_Size`	`size_t`	Number of elements currently constructed and valid in the container.
`m_Capacity`	`size_t`	Maximum size of the current memory block before requiring reallocation.
`push_back()`	`void`	Adds an element. Triggers the growth mechanic if `m_Size == m_Capacity`.
`operator[]`	`T&`	O(1) direct memory access, without bounds checking (for maximum performance).

2. Reallocation Mechanic (ReAlloc)

The main bottleneck of dynamic structures lies in system allocation calls. The growth strategy must be optimized:

- **Geometric Growth:** When the vector is full, allocate a new block with a capacity equal to `m_Capacity * 1.5` (or `2.0`). Never grow element by element.
- **Memory Transfer:** Use move semantics (`std::move`) to transfer objects from the old block to the new one without triggering expensive deep copies.
- **Cleanup:** Explicitly destroy old objects and free the original memory block via `delete[]`.

3. Strict Implementation: The Rule of Five

To guarantee the total absence of memory leaks or double frees, the C++ standard requires the explicit implementation of these five fundamental methods:

1. Destructor

Called automatically at the end of scope. Must invoke `delete[] m_Data;` to free all memory allocated to the array.

2. Copy Constructor

Triggered by `Vector a = b;`. Must allocate a new distinct memory block and copy elements from the source one by one.

3. Copy Assignment Operator (=)

Triggered by `a = b;`. Must check for self-assignment (`this == &b`), clean its own memory, then copy data from the source.

4. Move Constructor

Triggered with temporary objects (R-values). Steals the `m_Data` pointer from the source, then sets the source to `nullptr` to prevent its destructor from freeing the memory.

5. Move Assignment Operator

Checks for self-assignment, frees the current object's memory, takes ownership of the source's pointer, and resets the source to a valid but empty state (size 0, null pointer).

4. Validation Protocol (Micro-Demo)

The project will only be considered complete when the following two criteria are validated by the executable generated in the repository.

CRITERION 1: PERFORMANCE (BENCHMARK)

Execution time measured via <chrono> for 1,000,000 push_back calls compared to std::vector. The goal is to achieve similar or lower execution time.

CRITERION 2: MEMORY LEAK PROOF

```
> valgrind --leak-check=full ./vector_test
==12345== All heap blocks were freed -- no leaks are possible
```